Documentation Technique Track Loader

01.04.2026

—

Axel **NICOLAS**

Sommaire

Sommaire **2**
Introduction **4**
Fonctionnalités principales 4
Technologies utilisées 4
Pourquoi la musique ? 4
Architecture du Projet **5**
Structure des dossiers 5
Installation du projet **7**
Télécharger les ressources 7
Télécharger le projet 7
Depuis Moodle 7
Depuis GitHub 7
Télécharger les outils nécessaire 8
XAMPP 8
MySQL (via phpMyAdmin) 8
Configurer et lancer le projet 9
Importer la base de données 9
Configurer la connexion Xampp 9
Configurer les variables d'environnements 9
Lancer le projet 9
Modélisation des données **11**
Description des tables 11
Table : album 11
Table : user 12
Table : panier 12
Table : historique_achat 12
Liste des Endpoints API **14**
Statut de l'API 14
Genres & Artistes 14
Albums 14
Filtres disponibles sur GET /api/albums 14
Recherche globale 15
Utilisateur 15
Panier 16
Historique d'achats 16
Points Techniques **16**

Gestion du Token..... 16
Composition du Token..... 16
Processus de génération..... 17
Stockage..... 17
Validation..... 18
Sécurité..... 19
Recherche “Floue”..... 20
Principe de fonctionnement..... 20
Recherche multi-mots..... 20
Construction SQL..... 21
Sécurité..... 21
Achat d’un élément..... 23
Ajout au panier..... 23
Consultation du panier..... 23
Suppression d’un album..... 24
Validation de l’achat..... 24
Historique d’achats..... 24

Introduction

Track Loader est une application web d'achat d'albums musicaux développée dans le cadre d'un projet de Bachelor à SUPINFO Caen. Elle permet aux utilisateurs de découvrir, rechercher et acheter des morceaux de musique, des albums et des artistes, le tout depuis une interface moderne et intuitive.

Fonctionnalités principales

- Parcourir et rechercher des albums par nom, genre, année ou artiste
- Consulter le détail d'un album (tracklist, prix, artiste, style musical)
- Créer un compte, se connecter et gérer son profil utilisateur
- Ajouter des albums à un panier et valider une commande (checkout)
- Consulter l'historique de ses achats après validation
- Accéder aux albums en tendance sur la page d'accueil

Technologies utilisées

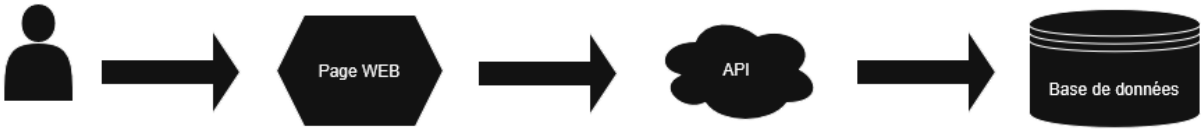
Couche	Technologie	Rôle
Frontend	PHP (pages dynamiques)	Affichage et interaction utilisateur
Backend / API	PHP (API REST)	Traitement des requêtes et logique métier
Base de données	MySQL	Stockage des utilisateurs, albums, panier, historique
Serveur local	XAMPP (Apache + MySQL)	Environnement de développement et d'exécution

Pourquoi la musique ?

Ce projet a été orienté autour de la musique par passion personnelle. La musique étant un domaine qui me tient particulièrement à cœur , ce thème a permis de s'investir pleinement dans la conception de l'application, aussi bien sur le plan technique que fonctionnel.

Architecture du Projet

L'application repose sur une architecture trois-tiers classique : une interface frontend en PHP qui communique avec une API REST (également en PHP), elle-même connectée à une base de données MySQL.



Structure des dossiers

Dossier / Fichier	Description
.htaccess	Configuration Apache : réécriture des URL (URL propres) et inclusion automatique de fichiers communs.
index.php	Page principale / point d'entrée du front-end.
/pages/	Pages PHP accessibles par l'utilisateur (album, artiste, panier, connexion, inscription...).
/include/	Fichiers inclus globalement sur toutes les pages : header.php, footer.php, notif.php.
/styles/	Feuilles de style CSS organisées en sous-dossiers : components/, layout/, pages/.
/assets/	Ressources statiques : images des artistes (icônes et bannières), image hero, logo.
/database/	Scripts SQL : init.sql (création des tables) et insert_data.sql (données initiales).
/scheme/	Diagramme de la base de données au format Looping (.lo1 / .loo).
/api/	L'API REST (architecture Routeur - Contrôleur - Service). Contient aussi le fichier .env (configuration, accès BDD, clés secrètes).
/api/config/	Configuration globale de l'API : connexion PDO (database.php) importée partout, et chargement des variables d'environnement depuis le .env (config.php).
/api/routes/	Définition des routes HTTP. Chaque fichier correspond à une ressource (ex. AlbumsRoutes.php, UserRoutes.php).

Dossier / Fichier	Description
/api/controller/	Contrôleurs : reçoivent la requête, appellent le service correspondant et retournent la réponse JSON.
/api/services/	Services : contiennent la logique métier et les requêtes vers la base de données.
/api/helpers/	Fonctions utilitaires génériques réutilisables dans toute l'API (helpers.php).
/api/utils/	Gestion des tokens : génération, validation et décodage (TokenUtils.php).

Installation du projet

Télécharger les ressources

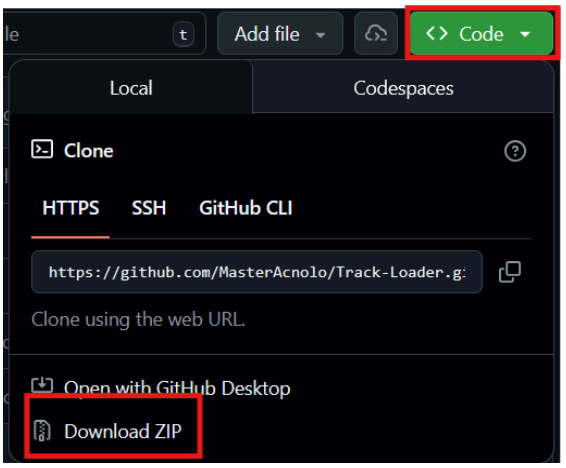
Télécharger le projet

Depuis Moodle

1. Ouvrez votre navigateur web (Firefox, Chrome, etc.).
2. Connectez-vous à votre plateforme Moodle .
3. Trouvez le dossier "Track Loader.zip".
4. Cliquez dessus pour le télécharger.
5. Le fichier se télécharge dans votre dossier "Téléchargements", vous pouvez le déplacer dans un endroit plus simple d'accès si besoin.

Depuis GitHub

1. Ouvrez votre navigateur web (Firefox, Chrome, etc.).
2. Accéder à la page suivante
3. Cliquez sur le bouton “Code” puis sur “Download Zip”



4. Le fichier se télécharge dans votre dossier "Téléchargements", vous pouvez le déplacer dans un endroit plus simple d'accès si besoin.

Télécharger les outils nécessaire

XAMPP

XAMPP est une distribution tout-en-un qui installe Apache, MySQL et PHP en une seule étape. C'est l'environnement recommandé pour faire fonctionner Track Loader en local.

- Téléchargez XAMPP depuis le site officiel : https://www.apachefriends.org
- Choisissez la version correspondant à votre système d'exploitation (Windows, macOS, Linux).
- Lancez l'installateur et suivez les étapes (laissez les options par défaut).
- Une fois installé, ouvrez le panneau de contrôle XAMPP.
- Démarrez les modules Apache et MySQL en cliquant sur « Start ».

MySQL (via phpMyAdmin)

MySQL est déjà inclus dans XAMPP. Vous y accédez via phpMyAdmin, l'interface web de gestion de base de données, disponible à l'adresse :

```
http://localhost/phpmyadmin
```

Configurer et lancer le projet

Importer la base de données

1. Ouvrez phpMyAdmin dans votre navigateur (http://localhost/phpmyadmin).
2. Créez une nouvelle base de données nommée trackloader.
3. Cliquez sur l'onglet « Importer ».
4. Sélectionnez le fichier init.sql fourni avec le projet.
5. Cliquez sur « Exécuter » les tables sont créées automatiquement.
6. Faites de mêmes pour le fichier “insert_data.sql”

Configurer la connexion Xampp

Ouvrez le fichier de configuration du projet (généralement config/config.php) et renseignez vos paramètres :

```
$host     = 'localhost';
$db_name  = 'trackloader';
$username = 'root';       // utilisateur par défaut XAMPP
$password = '';           // vide par défaut sous XAMPP
```

Configurer les variables d’environnements

La connexion à la base de données et les paramètres sensibles de l'application sont centralisés dans un fichier .env situé à la racine du dossier /api. Ouvrez ce fichier et adaptez les valeurs selon votre configuration locale :

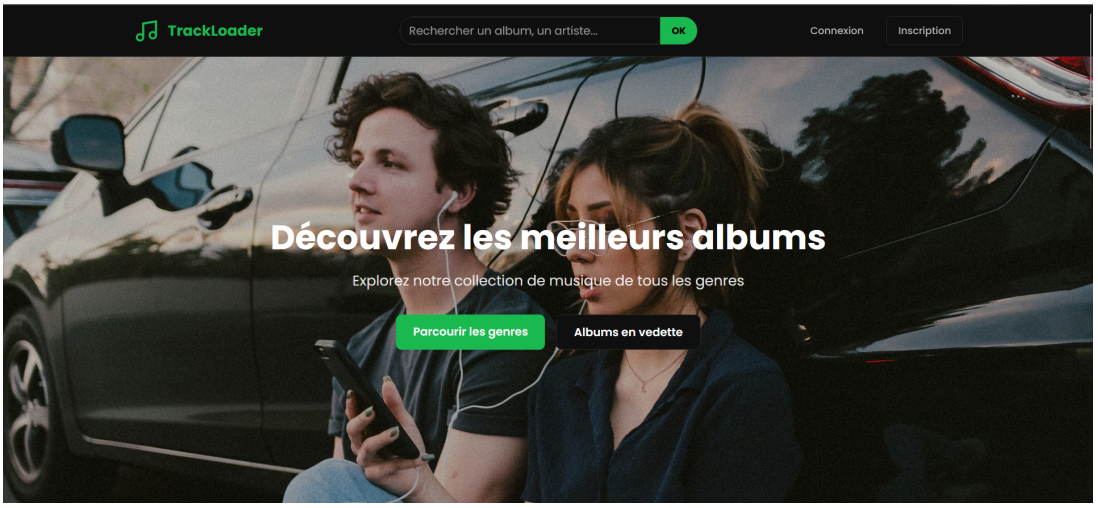
```
DB_HOST='localhost'
DB_NAME='trackloader' // Le nom de la base de données (défaut: trackloader)
DB_USER= 'USERNAME' // Le nom de l’utilisateur de la BDD
DB_PASS= 'PASSWORD' // Le mot de passe de l’utilisateur de la BDD

TOKEN_KEY= 'MOT ALEATOIRE' // Servira à encoder/ décoder le TOKEN
BASE_URL=http://localhost/Track-Loader
```

Lancer le projet

- Copiez le dossier du projet dans le répertoire htdocs de XAMPP C:\xampp\htdocs\track-loader
- Assurez-vous qu'Apache et MySQL sont bien démarrés dans le panneau XAMPP.
- Ouvrez votre navigateur et accédez à :

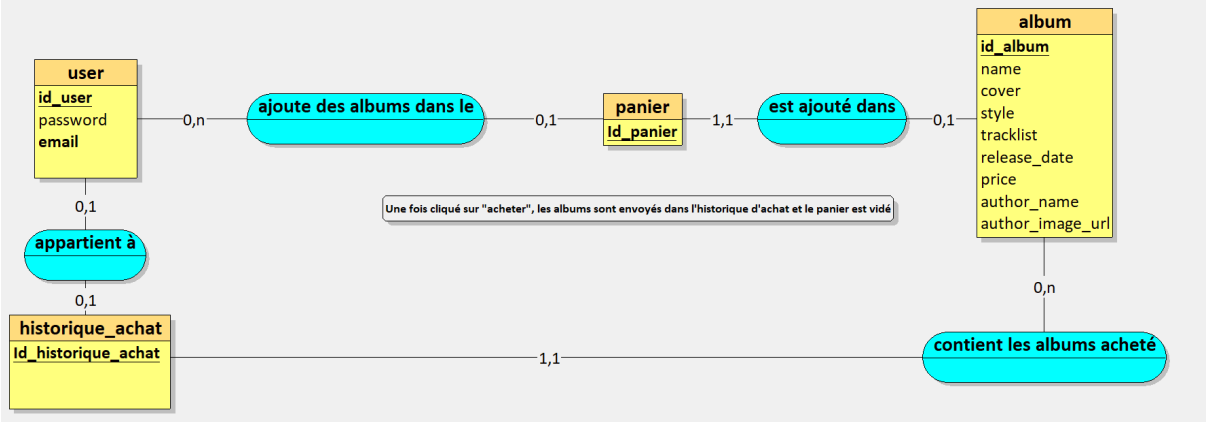
```
http://localhost/track-loader
```

L'application est maintenant accessible et prête à être utilisée.



Modélisation des données

La base de données de Track Loader repose sur quatre tables principales. Le diagramme ci-dessous (réalisé avec Looping) illustre les relations entre ces entités.



Description des tables

Table : album

Champ	Type	Contrainte	Description
id_album	INT	PK	Identifiant unique de l'album (auto-incrémenté)
name	VARCHAR(255)		Titre de l'album
cover	VARCHAR(255)		URL de l'image de couverture
style	VARCHAR(50)		Genre musical (ex : Rock, Jazz, Hip-Hop)
tracklist	TEXT		Liste des titres de l'album (format texte)
release_date	DATE		Date de sortie de l'album
price	INT		Prix de l'album en centimes (ou unité monétaire)
author_name	VARCHAR(255)		Nom de l'artiste ou du groupe
author_image_url	VARCHAR(255)		URL de la photo de l'artiste

Table : user

Champ	Type	Contrainte	Description
id_user	INT	PK	Identifiant unique de l'utilisateur (auto-incrémenté)
email	VARCHAR(255)	UNIQUE NOT NULL	Adresse e-mail qui sert d'identifiant de connexion
password	VARCHAR(255)	NOT NULL	Mot de passe hashé (bcrypt recommandé)

Table : panier

Représente les albums ajoutés au panier d'un utilisateur avant validation de la commande.

Champ	Type	Contrainte	Description
id_panier	INT	PK	Identifiant unique de l'entrée panier
id_user	INT	FK - user	Référence à l'utilisateur propriétaire du panier
id_album	INT	FK - album	Référence à l'album ajouté au panier
-	UNIQUE(id_user, id_album)		Un album ne peut être ajouté qu'une fois par utilisateur

Table : historique_achat

Enregistre chaque achat validé. Lors du checkout, les albums du panier sont transférés dans cette table et le panier est vidé. La suppression en cascade est activée sur id_user et id_album.

Champ	Type	Contrainte	Description
id_historique_achat	INT	PK	Identifiant unique de l'achat
id_user	INT	FK -> user	Utilisateur ayant effectué l'achat

Champ	Type	Contrainte	Description
id_album	INT	FK -> album	Album acheté
purchase_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date et heure de l'achat (automatique)

Note : les deux clés étrangères (id_user et id_album) sont définies avec ON DELETE CASCADE, ce qui signifie que la suppression d'un utilisateur ou d'un album entraîne automatiquement la suppression des entrées liées dans cette table.

Liste des Endpoints API

L'API REST de Track Loader expose l'ensemble de ses ressources via des endpoints HTTP. Toutes les réponses sont au format JSON. Les endpoints protégés nécessitent que l'utilisateur soit authentifié (session active).

Le projet n'intègre pas encore de versioning API (ex. /api/v1/) car il s'agit actuellement d'une mono-version destinée à un usage unique dans le cadre du projet. L'ajout d'un versioning serait pertinent en cas d'évolution ou de maintenance multi-version."

Statut de l'API

Méthode	Endpoint	Description
GET	/api/status	Vérifie que l'API est opérationnelle - retourne un statut 200 OK

Genres & Artistes

Méthode	Endpoint	Description
GET	/api/genres	Retourne la liste de tous les genres musicaux disponibles
GET	/api/artist	Retourne la liste de tous les artistes présents en base

Albums

Méthode	Endpoint	Description
GET	/api/albums	Liste tous les albums
GET	/api/albums/:ID	Retourne le détail d'un album par son identifiant
GET	/api/albums/trending	Retourne la liste des albums en tendance

Filtres disponibles sur GET /api/albums

Les paramètres suivants peuvent être passés en query string et combinés entre eux :

Paramètre	Exemple	Description
?name=XXX	?name=dark	Recherche par nom d'album (recherche approximative acceptée)
?genre=GENRE	?genre=Rock	Filtrer par genre musical
?annee=ANNEE	?annee=2020	Filtrer par année de sortie
?artiste=ARTISTE	?artiste=nf	Recherche par nom d'artiste (approximatif)
Combinaison	?genre=Rock-Alternatif&annee=2020	Tous les paramètres sont combinables

Recherche globale

Méthode	Endpoint	Description
GET	/api/search?q=QUERY	Recherche globale sur le nom d'album et le nom d'artiste

Utilisateur

Méthode	Endpoint	Description
POST	/api/user/login	Authentifie l'utilisateur (email + mot de passe). Ouvre une session.
POST	/api/user/register	Crée un nouveau compte utilisateur
POST	/api/user/password	Modifie le mot de passe de l'utilisateur connecté
GET	/api/user/logout	Déconnecte l'utilisateur et détruit la session
GET	/api/user/delete	Supprime le compte utilisateur (suppression cascade active sur panier et historique)

Panier

Méthode	Endpoint	Description
GET	/api/panier	Récupère la liste des albums dans le panier de l'utilisateur connecté
POST	/api/panier/:ID	Ajoute l'album identifié par :ID au panier
DELETE	/api/panier/:ID	Retire l'album identifié par :ID du panier
POST	/api/panier/checkout	Valide le panier : déplace les albums vers l'historique et vide le panier

Historique d'achats

Méthode	Endpoint	Description
GET	/api/historique	Récupère l'historique complet des achats de l'utilisateur connecté

Note : les endpoints du panier et de l'historique requièrent une session utilisateur active. En l'absence de session, l'API retourne une erreur 401 (non autorisé).

Points Techniques

Gestion du Token

Track Loader utilise un token pour maintenir l'authentification utilisateur entre les différentes pages de l'application. Ce token permet de vérifier qu'un utilisateur est bien connecté avant d'autoriser certaines actions sensibles comme l'accès au panier, à l'historique d'achats ou à la modification du mot de passe.

Composition du Token

Le token contient deux informations principales :

- l'adresse e-mail de l'utilisateur ;
- une date d'expiration.

La date d'expiration est fixée à 24 heures après la génération du token :

- time() + 86400

Le contenu est d'abord structuré sous forme de données encodées, puis chiffré avant stockage.

Processus de génération

```
function createToken($data, $config) {
   $payload = base64_encode(json_encode($data));
   $iv = random_bytes(16);
   $encrypted = openssl_encrypt($payload, 'AES-256-CBC', $config['TOKEN_KEY'], 0,
$iv);
   return base64_encode($iv . $encrypted);
}
```

Le mécanisme suit les étapes suivantes :

1. Création d'un payload contenant l'email utilisateur et la date d'expiration.
2. Encodage du payload en Base64.
3. Génération d'un vecteur d'initialisation aléatoire (IV).

4. Chiffrement avec l'algorithme AES-256-CBC via OpenSSL.
5. Concaténation de l'IV avec les données chiffrées.
6. Encodage final en Base64 pour obtenir le token exploitable.

La clé de chiffrement utilisée provient du fichier .env : TOKEN_KEY

Stockage

Une fois généré, le token est stocké dans un cookie navigateur ou dans la session PHP. Selon si l'on souhaite rester connecté ou non. Cela permet de restaurer automatiquement la session utilisateur lors de la navigation.

Validation

```
function verifyToken($token, $config) {
    $decoded = base64_decode($token);
    if (!$decoded || strlen($decoded) < 16) {
        notification('error', 'Token invalide ou expiré. Veuillez vous reconnecter.');
        exit;
    }

    $iv = substr($decoded, 0, 16);
    $encrypted = substr($decoded, 16);

    if (strlen($iv) !== 16) {
        notification('error', 'Token invalide ou expiré. Veuillez vous reconnecter.');
        exit;
    }

    $decrypted = openssl_decrypt($encrypted, 'AES-256-CBC', $config['TOKEN_KEY'],
0, $iv);
    if (!$decrypted) {
        notification('error', 'Token invalide ou expiré. Veuillez vous reconnecter.');
        exit;
    }

    $payload = json_decode(base64_decode($decrypted), true);
```

```
    if (!$payload || !isset($payload['exp'], $payload['email'])) {
        notification('error', 'Token invalide ou expiré. Veuillez vous reconnecter.');
        exit;
    }

    if ($payload['exp'] < time()) {
        notification('error', 'Session expirée. Veuillez vous reconnecter.');
        exit;
    }

    return $payload;
}
```

À chaque requête nécessitant une authentification :

1. le token est récupéré ;
2. l'IV est extrait ;
3. les données sont déchiffrées ;
4. le contenu est décodé ;
5. La date d'expiration est vérifiée.

Si le token est invalide ou expiré, l'utilisateur est considéré comme non authentifié.

Sécurité

Le chiffrement AES-256-CBC a été retenu car il permet :

- un chiffrement robuste ;
- l'utilisation d'un IV différent à chaque génération ;
- une meilleure protection des données sensibles stockées côté client.

La sécurité du système repose également sur la confidentialité de la clé TOKEN_KEY, qui ne doit jamais être exposée publiquement.

Recherche “Floue”

Track Loader utilise une recherche dite floue pour permettre à l’utilisateur de retrouver un album ou un artiste sans avoir besoin d’écrire exactement le nom complet.

Le principe consiste à rechercher des correspondances partielles dans :

- le nom de l’album ;
- le nom de l’artiste.

Cela permet par exemple de retrouver un album même si seul un mot partiel est saisi.

Principe de fonctionnement

La recherche repose sur l’opérateur SQL LIKE avec des jokers %.

Exemple :

- une recherche sur The permettra de trouver :
    - **The** Search
    - Hybrid **The**ory
    - **The**rapy Session

Chaque mot recherché est transformé en condition SQL :

%mot%

Cela signifie que le mot peut apparaître n’importe où dans le texte.

Recherche multi-mots

Lorsque plusieurs mots sont saisis, la requête est découpée mot par mot.
Exemple :

Linkin Meteora

Devient :

- Linkin
- Meteora

Chaque mot génère une condition indépendante.

Un album est retenu uniquement si tous les mots sont retrouvés, soit dans le nom de l'album, soit dans le nom de l'artiste.

Construction SQL

Pour chaque mot :

- une condition est créée sur name
- une condition est créée sur author_name

Les conditions sont ensuite reliées avec AND.

```
WHERE (name LIKE '%Linkin%' OR author_name LIKE '%Linkin%')
AND (name LIKE '%Meteora%' OR author_name LIKE '%Meteora%')
```

Cela permet :

- de rechercher plusieurs termes ;
- de combiner album et artiste ;
- d'obtenir des résultats plus précis.

Sécurité

La requête utilise des requêtes préparées PDO avec paramètres nommés. Cela évite les injections SQL.

Chaque mot devient un paramètre indépendant :

- **:word0**
- **:word1**
- **:word2**

Exemple de logique interne

1. suppression des espaces inutiles ;

2. découpage de la recherche en plusieurs mots ;
3. création dynamique des conditions SQL ;
4. exécution avec PDO ;
5. retour JSON des albums trouvés.

Cas particuliers

- une recherche vide retourne un tableau vide ;
- plusieurs espaces sont automatiquement ignorés ;
- La recherche n’est pas sensible à la casse selon la configuration MySQL.

Achat d'un élément

Le système d'achat de Track Loader repose sur deux tables distinctes :

- **panier** : contient les albums ajoutés avant validation ;
- **historique_achat** : contient les albums achetés définitivement.

Le panier agit comme une zone temporaire de préparation avant validation de la commande.

Ajout au panier

Lorsqu'un utilisateur ajoute un album :

- son identifiant utilisateur est récupéré ;
- l'identifiant de l'album est transmis ;
- une insertion est effectuée dans la table **panier**.

Une contrainte d'unicité empêche qu'un même album soit ajouté plusieurs fois

```
UNIQUE(id_user, id_album)
```

Ainsi :

- un album déjà présent n'est pas dupliqué ;
- La cohérence du panier est conservée.

Consultation du panier

Le panier est récupéré en reliant :

- la table **panier**
- la table **album**

Cela permet d'afficher directement :

- le titre ;
- le prix ;
- la couverture ;
- l'artiste ;
- le style musical.

Suppression d'un album

Lorsqu'un album est retiré :

- la suppression cible uniquement :
  - l'utilisateur connecté ;
  - l'album sélectionné.

Cela évite toute suppression accidentelle sur d'autres utilisateurs.

Validation de l'achat

Lors du checkout :

1. les albums présents dans le panier sont récupérés ;
2. chaque album est inséré dans **historique_achat** ;
3. Le panier est vidé.

Le transfert utilise une transaction SQL.

Principe :

```
BEGIN;
INSERT INTO historique_achat (...);
DELETE FROM panier (...);
COMMIT;
```

Si une erreur survient :

```
ROLLBACK;
```

Cela garantit :

- qu'un achat est entièrement validé ;
- ou totalement annulé.

Aucun état intermédiaire n'est conservé.

Historique d'achats

Une fois l'achat confirmé :

- les albums apparaissent dans l'historique personnel de l'utilisateur ;

- La date d'achat est automatiquement enregistrée.

Le champ utilisé :

```
purchase_date DATETIME DEFAULT CURRENT_TIMESTAMP
```

Chaque achat conserve donc :

- l'utilisateur
- l'album
- la date exacte.

Intégrité des données

Les clés étrangères utilisent `ON DELETE CASCADE.`

Cela signifie :

- si un utilisateur est supprimé, son panier et son historique sont supprimés ;
- Si un album est supprimé, les références associées disparaissent automatiquement.